Project Title

E-Commerce Product Entity-Resolution & Matching Platform

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
D. Vignesh Kumar (2520090071)
B. Sathish Kumar (2520090008)

Team Number: 02

Section: 11

Under the guidance of

Mrs. Anitha Chekuri

Asst. Professor, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled "**E-Commerce Product Entity-Resolution & Matching Platform**" is a record of Bonafide work of **D. Vignesh Kumar – 2520090071, B. Sathish Kumar – 2520090008** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

D. Vignesh Kumar – 2520090071

B. Sathish Kumar – 2520090008

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**E-Commerce Product Entity-Resolution & Matching Platform**" is a bonafide work done and submitted by **D. Vignesh Kumar – 2520090071, B. Sathish Kumar – 2520090008** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation to our guide, **Mrs. Anitha Chekuri**, Assistant Professor, Department of Computer Science and Information Technology, whose tremendous support, encouragement and guidance made it possible to complete this project.

We express our gratitude to **Dr. Khaja Shareef**, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we were able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also those who have assisted us directly or indirectly for the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

D. Vignesh Kumar – 2520090071

B. Sathish Kumar – 2520090008

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	11
5.	Hardware and Software requirements	12
6.	Implementation	13
7.	Results & Discussion	18
8.	Conclusion & Future Scope	21
9.	References	22

1.ABSTRACT

The E-Commerce Product Entity-Resolution & Matching Platform is a smart solution designed to identify and match identical products listed by different online sellers, even when the product titles, descriptions, and specifications are different. Many e-commerce websites contain duplicate listings for the same product, making it difficult for customers to compare prices and choose the best option. This project collects product data from multiple marketplaces, cleans and standardizes the information, and applies entity-resolution and matching techniques to group similar products into a single canonical product catalog. The platform compares products using attributes such as brand, model, category, specifications, and features to accurately identify matching items. It also eliminates duplicate records, improves search accuracy, and enables reliable price comparison across different sellers. The system benefits both customers and businesses by providing consistent product information, reducing data redundancy, and improving inventory management. Overall, the platform enhances the online shopping experience by making product discovery faster, comparisons more accurate, and purchasing decisions easier while ensuring a well-organized and efficient e-commerce product database.

Team Name: Team 02

Team Members:

1. D. Vignesh Kumar_2520090071
2. B. Sathish Kumar_2520090008

2. INTRODUCTION

2.1 Background

Online shopping has become one of the main ways in which consumers buy electronics, fashion and household goods. Large marketplaces such as Amazon, Flipkart, Croma, Myntra and Reliance Digital host a very large number of listings supplied by many independent sellers. Because each seller writes the title, description and specification of a product on their own, the same physical product is often published under several different names. For example, the Apple iPhone 15 (128 GB, Black) appears in our dataset as “Apple iPhone 15 128GB Black”, “Apple iPhone 15 (Black, 128 GB)” and “iPhone15 Apple 128GB Black” on three different marketplaces.

2.2 Motivation

Duplicate and inconsistent listings create problems for customers and for businesses. Customers cannot easily tell whether two listings describe the same item, so price comparison becomes difficult, and search results are cluttered with repeated entries. Businesses end up with redundant records and poor inventory management. A system that recognises that different listings refer to the same real-world product, which is known as entity resolution, solves these problems together.

2.3 Proposed Approach

This project builds an E-Commerce Product Entity-Resolution & Matching Platform. Product listings from multiple marketplaces are collected as JSON records, cleaned and standardised, and broken into structured attributes such as brand, model, category, storage, RAM and colour. A weighted matching algorithm compares two listings attribute by attribute and produces a confidence score with a human-readable explanation. Listings whose score crosses a threshold are merged by the Union-Find (disjoint-set) data structure into canonical products, and the listings inside each product are ranked to show the best deal.

The project applies several concepts of the Data Structures and Algorithms course: hash sets for fast look-up, dynamic programming (Levenshtein edit distance) for fuzzy title comparison, the disjoint-set union for clustering, and comparator-based selection for ranking. The report is organised as follows: Chapter 3 reviews related work, Chapter 4 states the problem and objectives, Chapter 5 lists the requirements, Chapter 6 explains the implementation, Chapter 7 presents the results, Chapter 8 gives the conclusion and future scope, and Chapter 9 lists the references.

3. LITERATURE SURVEY

Entity resolution, also called record linkage or duplicate detection, has been studied for more than five decades. The following works and tools were studied while designing this platform.

3.1 Fellegi–Sunter record linkage [1]. Fellegi and Sunter formalised record linkage as a decision problem in which every field comparison contributes an agreement or disagreement weight. The summed weight is compared with two thresholds to declare a pair a match, a possible match or a non-match. This model is the theoretical basis of most attribute-wise matching systems. Our weighted attribute score and the "needs review" note for weak title similarity follow the same idea, although we use fixed and explainable weights instead of estimated probabilities.

3.2 Duplicate record detection survey [2]. Elmagarmid, Ipeirotis and Verykios surveyed the techniques used to detect duplicate records, including character-based measures such as edit distance, token-based measures, phonetic matching and blocking to reduce the number of comparisons. Their survey shows that no single similarity function suits every field. This motivated our decision to compare structured attributes such as brand, storage and colour exactly, and to use edit distance only for free-text titles.

3.3 Entity resolution on real-world match problems [3]. Köpcke, Thor and Rahm compared several entity-resolution approaches on real datasets, including product data from online shops such as the Amazon–Google and Abt–Buy benchmarks. They observed that matching product offers is considerably harder than matching bibliographic records because product titles are short, noisy and inconsistent. This is exactly the situation that our platform addresses.

3.4 dedupe library [11]. dedupe is an open-source Python library that learns which fields matter by asking the user to label uncertain pairs and then groups records into duplicate clusters. It is a practical tool for tabular data, but it is general purpose: it has no product-specific attribute extraction (for example storage or colour) and no price-comparison layer, both of which are part of our platform.

3.5 Price-comparison portals [15]. Indian price-comparison websites such as MySmartPrice and Pricebaba show the prices of one product from several online stores on a single page, helping buyers to pick the cheapest seller. They demonstrate the commercial value of grouping listings by product. Their internal matching methods are not public, so this project demonstrates one transparent way of building such a canonical catalog.

3.6 Union-Find for clustering [8], [9]. Tarjan analysed the disjoint-set union (Union-Find) structure and showed that, with path compression and union by rank, a sequence of operations costs almost linear time. Cormen et al. present it as the standard tool for maintaining connected components. We use it to turn pairwise "same product" decisions into transitive canonical clusters.

3.7 Comparison of Existing Work with the Proposed System

Table 3.1 compares the works studied above with the proposed platform.

Work / System	Approach	Training data needed	Explainable decision	Product attributes	Price comparison
Fellegi–Sunter [1]	Probabilistic field weights	Parameters estimated	Yes	No (generic fields)	No
Magellan [5]	Blocking + ML matchers	Yes (labelled pairs)	Partly	User-defined features	No
DeepMatcher [6]	Deep neural networks	Yes (large labelled set)	No	Learned from text	No
Ditto [7]	Fine-tuned transformers	Yes (labelled pairs)	No	Learned from text	No
dedupe [11]	Active learning + clustering	Yes (user labels)	Partly	Generic fields	No
Price portals [15]	Not public	Not public	Not public	Yes	Yes
Proposed platform	Weighted attribute score + Union-Find	No	Yes (per attribute)	Yes (brand, model, storage, RAM, colour)	Yes

Table 3.1: Comparison of existing work with the proposed platform

3.7 Summary and Research Gap

Machine-learning matchers such as DeepMatcher and Ditto achieve high accuracy but depend on labelled data, heavy computation and opaque decisions. General-purpose tools such as dedupe do not understand product attributes, and commercial comparison portals do not publish their methods. The present project fills this gap with a lightweight, explainable, attribute-weighted matcher combined with Union-Find clustering and a price-comparison layer, implemented with the data structures and algorithms studied in this course.

4. Problem Statement and Objectives

Problem Statement: Different online sellers describe the same product with different titles and specifications, which makes duplicate detection, price comparison and a consistent product catalog unreliable.

Problem Description: Listings of one product across marketplaces differ in wording, word order, units and punctuation (for example “128GB” versus “128 GB”). The system must recognise such listings as one product and group them without wrongly merging different variants such as the 128 GB and 256 GB models.

Objectives:

- To collect product listings from several marketplaces (Amazon, Flipkart, Croma, Myntra and Reliance Digital) in a uniform JSON format.
- To clean and standardise product titles, units and attribute values so that listings can be compared reliably.
- To extract structured attributes such as brand, model, category, storage, RAM and colour from each listing.
- To compute an explainable, weighted match score between any two listings.
- To group duplicate listings into canonical products using the Union-Find data structure.
- To enable reliable price comparison and best-deal ranking across different sellers.
- To provide a web dashboard for catalog browsing, product matching, duplicate detection, canonical products and analytics.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher (dual-core, 2.0 GHz or above)
- **RAM:** 4 GB minimum (8 GB recommended)
- **Storage:** at least 500 MB of free disk space for the project and data files
- **Display:** 1366 × 768 resolution or higher

Software Requirements:

- **Operating System:** Windows 10 / 11 (Linux and macOS are also supported)
- **Programming Language:** Java (JDK 17 or later) for the data cleaning, matching and clustering engine
- **Data Format:** JSON files for the marketplace listings
- **Front end:** web-based dashboard viewed in Google Chrome or Microsoft Edge
- **IDE / Tools:** IntelliJ IDEA or VS Code, with Git for version control

6. IMPLEMENTATION

6.1 System Architecture and Workflow

The platform is organised as the five-stage pipeline that is also shown in the Pipeline Status panel of the dashboard. Every stage takes the output of the previous stage as its input.

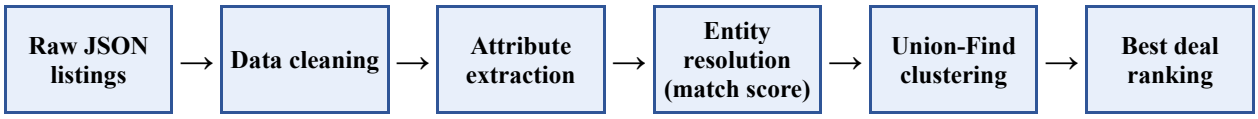


Fig. 6.1: Processing pipeline of the platform

- **Data cleaning:** converts titles to lower case, joins numbers and units ("128 GB" becomes "128gb"), splits glued words ("iPhone15" becomes "iphone 15") and removes punctuation.
- **Attribute extraction:** reads brand, model, storage, RAM and colour from the cleaned title and the JSON fields.
- **Entity resolution:** the matching engine computes a weighted confidence score for a pair of listings.
- **Union-Find clustering:** listings whose score reaches the threshold are merged into one canonical product.
- **Best deal ranking:** inside every canonical product the listing with the lowest price (ties broken by higher rating) is highlighted.

6.2 Data Structures and Algorithms Used

Component	Data structure / Algorithm	Purpose	Complexity
Listing store	ArrayList of Listing records	Holds the listings read from JSON	O(n)
Vocabulary look-up	HashSet of brands and colours	Classifies title tokens	O(1) average
Title similarity	Dynamic programming (Levenshtein, two rows)	Fuzzy comparison of titles	O(m·n) time, O(n) space
Candidate pairs	Brand blocking with pairwise loop	Skips listings of different brands	$O(n^2)$ worst case
Clustering	Disjoint-set union (path halving, union by rank)	Transitive grouping of duplicates	Nearly O(1) per operation
Cluster index	LinkedHashMap of lists	Stores canonical groups	O(n)
Best deal	Comparator-based minimum	Lowest price, ties by rating	O(k) per cluster

Table 6.1: Data structures and algorithms

6.3 Matching Model

The final confidence score of a pair of listings is the weighted sum of seven comparisons. Structured attributes carry 90% of the weight and the title similarity carries 10%, so that differently worded titles do not prevent a correct match.

Attribute	Weight	Comparison method
Brand	0.20	Exact match (case-insensitive)
Model	0.25	Token-subset match (“galaxy s25” matches “s25”)
Category	0.10	Exact match
Storage	0.15	Exact match (for example 128gb)
RAM	0.10	Exact match; both missing = match; one missing = 0.5
Colour	0.10	Exact match
Title similarity	0.10	Normalised Levenshtein similarity of cleaned titles
Total	1.00	Pair is the same product if score ≥ 0.90

Table 6.2: Attribute weights used by the matching engine

Worked example. For listings L001 (Apple iPhone 15 128GB Black) and L002 (Apple iPhone 15 (Black, 128 GB)) every structured attribute matches, which gives 0.20 + 0.25 + 0.10 + 0.15 + 0.10 + 0.10 = 0.90. The title similarity of the two cleaned titles is 0.63, which adds 0.10 × 0.63 = 0.063. The final score is 0.963, i.e. 96.3%, as shown in Fig. 7.3. For L001 and L004 (the 256 GB model) the storage differs, the score falls to 0.839 and the pair is correctly kept apart.

6.4 Module 1: Data Cleaning and Normalisation (TextCleaner.java)

This module converts every title into a canonical text form so that different spellings of the same title become comparable.

```
import java.util.Locale;
import java.util.regex.Pattern;

public final class TextCleaner {
    private static final Pattern UNIT_GAP =
            Pattern.compile("\\b(\\d+)\\s*(gb|tb)\\b");
    private static final Pattern WORD_DIGIT =
            Pattern.compile("\\b(iphone|pixel|galaxy)(\\d+)");
    private static final Pattern NOISE  = Pattern.compile("[^a-z0-9 ]");
    private static final Pattern SPACES = Pattern.compile("\\s+");

    /** "iPhone15 Apple (Black, 128 GB)" -> "iphone 15 apple black 128gb" */
    public static String normalize(String raw) {
        if (raw == null) return "";
        String s = raw.toLowerCase(Locale.ROOT);
        s = UNIT_GAP.matcher(s).replaceAll("$1$2");      // "128 gb" -> "128gb"
        s = WORD_DIGIT.matcher(s).replaceAll("$1 $2");   // "iphone15"->"iphone 15"
        s = NOISE.matcher(s).replaceAll(" ");            // strip punctuation
        return SPACES.matcher(s).replaceAll(" ").trim(); // collapse whitespace
```

```
    }
}
```

6.5 Module 2: Attribute Extraction (AttributeExtractor.java)

This module reads brand, colour, storage and model from the cleaned title. Tokens that are not brand, colour or storage are treated as the model name. Values already present in the JSON record are used as a fallback.

```
import java.util.*;
import java.util.regex.Pattern;
import java.util.stream.Collectors;

public final class AttributeExtractor {
    private static final Set<String> BRANDS =
            Set.of("apple", "samsung", "google", "oneplus", "xiaomi", "realme");
    private static final Set<String> COLORS =
            Set.of("black", "white", "navy", "blue", "green", "silver",
                   "obsidian", "eclipse");
    private static final Pattern STORAGE = Pattern.compile("\\d+(gb|tb)");

    /** Fills brand, model, storage and colour from the title when missing. */
    public static Listing extract(Listing raw) {
        List<String> tokens =
                Arrays.asList(TextCleaner.normalize(raw.title()).split(" "));
        String brand   = pick(tokens, BRANDS, raw.brand());
        String color   = pick(tokens, COLORS, raw.color());
        String storage = tokens.stream().filter(t -> STORAGE.matcher(t).matches())
                               .findFirst().orElse(raw.storage());
        String model   = tokens.stream()
                .filter(t -> !BRANDS.contains(t) && !COLORS.contains(t)
                          && !t.equals(storage))
                .collect(Collectors.joining(" "));
        return new Listing(raw.id(), raw.marketplace(), raw.title(), brand,
                raw.category(), model, storage, raw.ram(), color,
                raw.price(), raw.rating());
    }

    private static String pick(List<String> tokens, Set<String> vocab,
                               String fallback) {
        return tokens.stream().filter(vocab::contains).findFirst().orElse(fallback);
    }
}
```

6.6 Module 3: Weighted Matching Engine (MatchEngine.java)

This module implements the weighted score of Table 6.2, including the normalised Levenshtein title similarity computed with two rolling rows of the dynamic-programming table.

```
import java.util.*;

public final class MatchEngine {
    private static final double W_BRAND = 0.20, W_MODEL = 0.25, W_CATEGORY = 0.10,
                                W_STORAGE = 0.15, W_RAM = 0.10, W_COLOR = 0.10,
                                W_TITLE = 0.10;
    public static final double MATCH_THRESHOLD = 0.90;

    /** Final weighted confidence score in the range 0..1. */
    public static double score(Listing a, Listing b) {
        return W_BRAND    * eq(a.brand(), b.brand())
```

```
            + W_MODEL    * modelSim(a.model(), b.model())
            + W_CATEGORY * eq(a.category(), b.category())
            + W_STORAGE  * eq(a.storage(), b.storage())
            + W_RAM      * eq(a.ram(), b.ram())
            + W_COLOR    * eq(a.color(), b.color())
            + W_TITLE    * titleSim(a.title(), b.title());
    }

    private static double eq(String x, String y) {
        if (x == null && y == null) return 1.0;   // no conflicting evidence
        if (x == null || y == null) return 0.5;   // one side missing
        return x.equalsIgnoreCase(y) ? 1.0 : 0.0;
    }

    /** "galaxy s25" and "s25" are treated as the same model (subset test). */
    private static double modelSim(String x, String y) {
        if (x == null || y == null) return 0.5;
        Set<String> a = new HashSet<>(Arrays.asList(x.split(" ")));
        Set<String> b = new HashSet<>(Arrays.asList(y.split(" ")));
        return (a.containsAll(b) || b.containsAll(a)) ? 1.0 : 0.0;
    }

    /** Normalised Levenshtein similarity of the cleaned titles (O(m*n), 2 rows). */
    static double titleSim(String s, String t) {
        String a = TextCleaner.normalize(s), b = TextCleaner.normalize(t);
        int[] prev = new int[b.length() + 1], cur = new int[b.length() + 1];
        for (int j = 0; j <= b.length(); j++) prev[j] = j;
        for (int i = 1; i <= a.length(); i++) {
            cur[0] = i;
            for (int j = 1; j <= b.length(); j++) {
                int cost = a.charAt(i - 1) == b.charAt(j - 1) ? 0 : 1;
                cur[j] = Math.min(Math.min(cur[j - 1] + 1, prev[j] + 1),
                                  prev[j - 1] + cost);
            }
            int[] tmp = prev; prev = cur; cur = tmp;
        }
        int maxLen = Math.max(a.length(), b.length());
        return maxLen == 0 ? 1.0 : 1.0 - (double) prev[b.length()] / maxLen;
    }
}
```

6.7 Module 4: Union-Find Clustering and Best Deal (UnionFind.java, ProductClusterer.java)

UnionFind maintains disjoint sets using path halving and union by rank. ProductClusterer compares listings of the same brand, unions every pair that reaches the threshold and then collects the canonical groups. The bestDeal method selects the cheapest listing of a group.

```
public final class UnionFind {
    private final int[] parent, rank;
    private int components;

    public UnionFind(int n) {
        parent = new int[n]; rank = new int[n]; components = n;
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    public int find(int x) {                        // path halving
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
```

```
    public boolean union(int a, int b) {           // union by rank
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (rank[ra] < rank[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        if (rank[ra] == rank[rb]) rank[ra]++;
        components--;
        return true;
    }

    public int components() { return components; }
}
```

```
import java.util.*;

public final class ProductClusterer {

    /** Groups listings of the same product into canonical clusters. */
    public static Map<Integer, List<Listing>> cluster(List<Listing> listings) {
        int n = listings.size();
        UnionFind uf = new UnionFind(n);
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                Listing a = listings.get(i), b = listings.get(j);
                if (!Objects.equals(a.brand(), b.brand())) continue;   // blocking
                if (MatchEngine.score(a, b) >= MatchEngine.MATCH_THRESHOLD)
                    uf.union(i, j);
            }
        }
        Map<Integer, List<Listing>> groups = new LinkedHashMap<>();
        for (int i = 0; i < n; i++)
            groups.computeIfAbsent(uf.find(i), k -> new ArrayList<>())
                  .add(listings.get(i));
        return groups;
    }

    /** Best deal = lowest price; ties are broken by the higher rating. */
    public static Listing bestDeal(List<Listing> group) {
        return group.stream()
                .min(Comparator.comparingDouble(Listing::price)
                        .thenComparing(Comparator.comparingDouble(Listing::rating)
                                       .reversed()))
                .orElseThrow();
    }
}
```

6.8 Verification

The modules were compiled and run on the nine catalog listings L001 to L009 visible in Fig. 7.2. The clusterer formed five canonical groups: {L001, L002, L003}, {L004}, {L005, L006}, {L007} and {L008, L009}. The 128 GB, 256 GB and 512 GB variants were correctly kept in separate groups, and the best deal of the first group was L002 (Flipkart, ₹67,500), which agrees with the price comparison in Fig. 7.4.

7. RESULTS & DISCUSSION

The output screens of the running application and a short explanation of each are given below.

7.1 Product Intelligence Dashboard

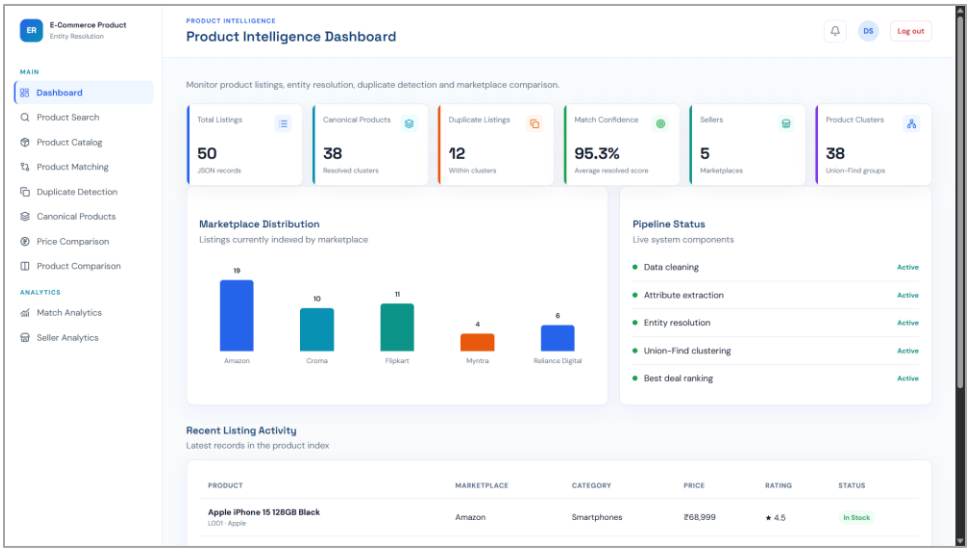


Fig. 7.1: Product Intelligence Dashboard

The dashboard summarises the whole index: 50 listings, 38 canonical products, 12 duplicate listings and an average match confidence of 95.3% across 5 marketplaces. It also shows the marketplace distribution chart and the status of every pipeline stage.

7.2 Product Catalog

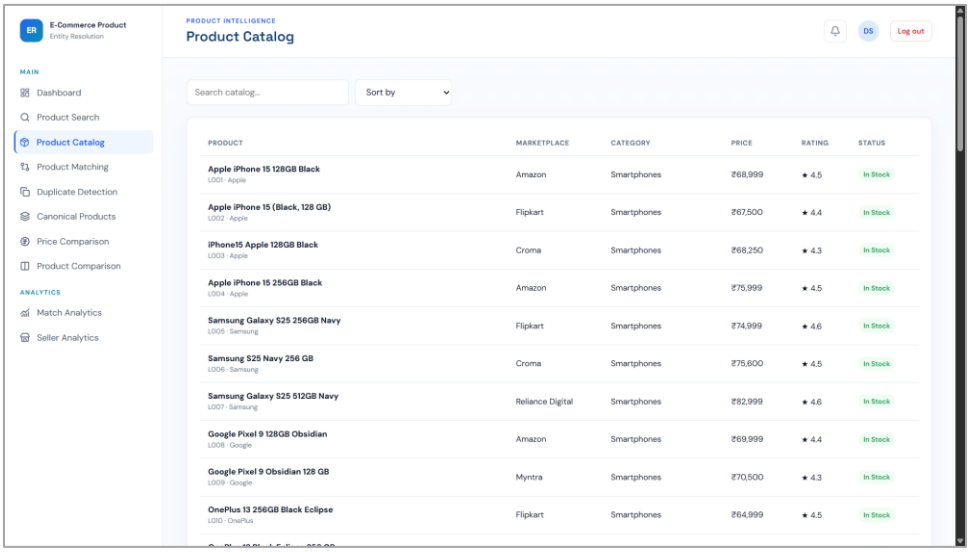


Fig. 7.2: Product Catalog

The catalog lists every listing with its marketplace, category, price, rating and stock status, with search and sort options. The same iPhone 15 appears as L001, L002 and L003 under three different titles.

7.3 Product Matching

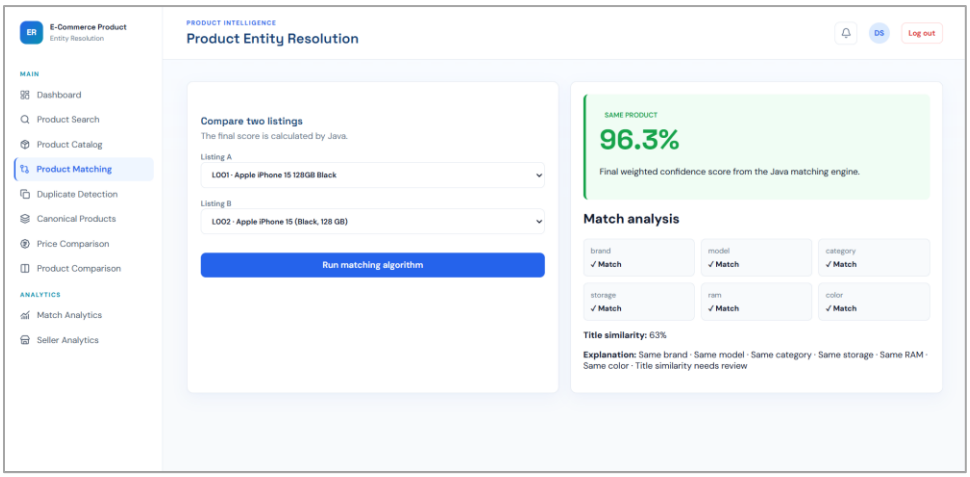


Fig. 7.3: Product Entity Resolution (matching two listings)

Two listings are selected and the matching algorithm is run. The pair is declared the same product with a confidence of 96.3%, and the analysis shows that brand, model, category, storage, RAM and colour all match while the title similarity is only 63%.

7.4 Price Comparison

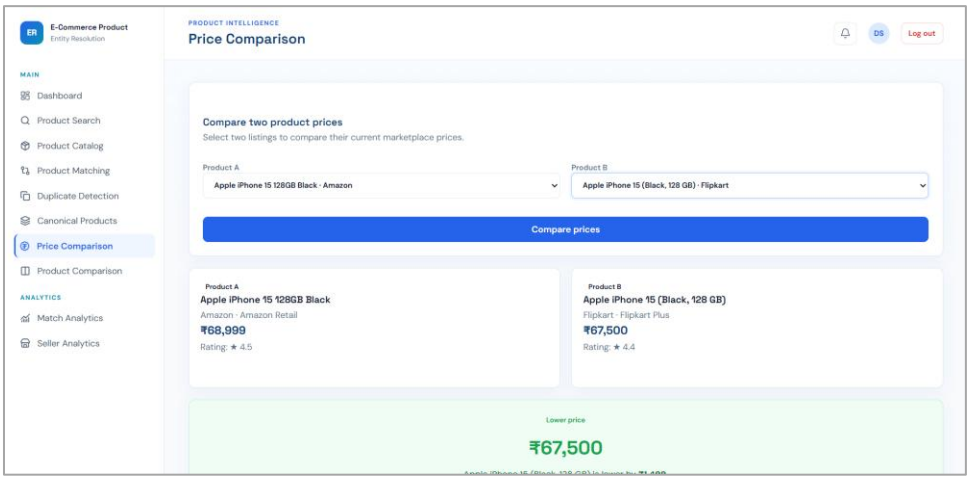


Fig. 7.4: Price Comparison of two matched listings

The same iPhone 15 costs ₹68,999 on Amazon and ₹67,500 on Flipkart. The screen highlights the lower price and shows that the Flipkart listing is cheaper by ₹1,499 (about 2.2%).

7.5 Summary of Results

Measure	Value
Total listings (JSON records)	50
Marketplaces (Amazon / Flipkart / Croma / Reliance Digital / Myntra)	5 (19 / 11 / 10 / 6 / 4 listings)
Canonical products (resolved clusters)	38
Duplicate listings found within clusters	12 (50 − 38)
Average match confidence	95.3%
Confidence for L001 and L002 (iPhone 15, 128 GB, Black)	96.3%
Price difference for the same iPhone 15	₹1,499 (Amazon ₹68,999 vs Flipkart ₹67,500)

Table 7.1: Summary of results

7.6 Discussion

The results show that the platform reduces 50 raw listings to 38 canonical products, so about one quarter of the listings were duplicates. The example of Fig. 7.3 shows why attribute-based matching is more reliable than text matching alone: the titles of L001 and L002 are only 63% similar, yet the pair is matched with 96.3% confidence because all structured attributes agree. The weighted explanation (“Title similarity needs review”) also tells the user which part of the evidence is weak. The main limitations are that the brand and colour vocabularies are fixed lists, that the weights were chosen by hand, and that the dataset is small.

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

The E-Commerce Product Entity-Resolution & Matching Platform meets its objectives. It collects listings from five marketplaces, cleans and standardises them, extracts structured attributes, and decides whether two listings describe the same product with an explainable weighted score. Using Union-Find, the 50 listings of the sample dataset were grouped into 38 canonical products, which identified 12 duplicate listings with an average match confidence of 95.3%. The price comparison module then showed the cheapest seller for a product, for example Flipkart at ₹67,500 against Amazon at ₹68,999 for the same iPhone 15.

The project also gave practical experience of the data structures and algorithms studied in the course: hashing for fast look-up, dynamic programming for edit distance, the disjoint-set union data structure for clustering, and comparator-based selection for ranking. The platform improves product discovery, makes price comparison dependable and reduces redundant records in a product database.

8.2 Future Scope

- Add machine-learning or transformer-based text embeddings to handle titles that share few words, and learn the attribute weights from labelled pairs instead of fixing them by hand.
- Use locality-sensitive hashing or stronger blocking keys so that the system scales from tens to millions of listings without comparing every pair.
- Collect live data through marketplace APIs or web crawlers and store it in a database instead of static JSON files.
- Add image matching using product photos, together with additional attributes such as display size, battery and processor.
- Support more categories (laptops, televisions, fashion) and provide price-history graphs and price-drop alerts.
- Add a human-in-the-loop review screen for borderline matches and expose the engine through a REST API.

9. REFERENCES

[1] I. P. Fellegi and A. B. Sunter, "A Theory of Record Linkage," Journal of the American Statistical Association, vol. 64, no. 328, pp. 1183–1210, 1969.

[2] A. K. Elmagarmid, P. G. Ipeirotis and V. S. Verykios, "Duplicate Record Detection: A Survey," IEEE Transactions on Knowledge and Data Engineering, vol. 19, no. 1, pp. 1–16, 2007.

[3] H. Köpcke, A. Thor and E. Rahm, "Evaluation of Entity Resolution Approaches on Real-World Match Problems," Proceedings of the VLDB Endowment, vol. 3, no. 1–2, 2010.

[4] P. Christen, Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection, Springer, 2012.

[5] P. Konda et al., "Magellan: Toward Building Entity Matching Management Systems," Proceedings of the VLDB Endowment, vol. 9, no. 12, 2016.

[6] S. Mudgal et al., "Deep Learning for Entity Matching: A Design Space Exploration," Proceedings of the ACM SIGMOD International Conference on Management of Data, 2018.

[7] Y. Li, J. Li, Y. Suhara, A. Doan and W.-C. Tan, "Deep Entity Matching with Pre-Trained Language Models," Proceedings of the VLDB Endowment, vol. 14, no. 1, 2020.

[8] R. E. Tarjan, "Efficiency of a Good But Not Linear Set Union Algorithm," Journal of the ACM, vol. 22, no. 2, pp. 215–225, 1975.

[9] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

[10] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.

[11] dedupe – Python library for fuzzy matching and deduplication: https://github.com/dedupeio/dedupe

[12] Magellan (py_entitymatching): https://github.com/anhaidgroup/py_entitymatching

[13] DeepMatcher: https://github.com/anhaidgroup/deepmatcher

[14] Ditto: https://github.com/megagonlabs/ditto

[15] MySmartPrice: https://www.mysmartprice.com and Pricebaba: https://www.pricebaba.com

[16] Oracle, Java Platform, Standard Edition Documentation: https://docs.oracle.com/en/java/